

INSuRE Fall 2024
Research Project Proposal
Scanning the Matrix: Harnessing Automation for Vulnerability Scans
(v1)

Technical Directors: Bre'Ona Williams
Jerome Lomax
Regis Wilson

The University of Texas at Dallas
Team 3

Nishanth Badim - Student Researcher - nxb220044@utdallas.edu
Harika Gattu - Student Researcher - hxg220030@utdallas.edu
Divya-Bala-Sri Gogineni - Student Researcher - dxg220035@utdallas.edu
Bhavana-Bramaram Gurram - Student Researcher - bxc220017@utdallas.edu
Savyasachi Kokkiralala - Student Researcher - sxc220272@utdallas.edu
Brian Ricks - Faculty Support - bwr031000@utdallas.edu

- PROBLEM STATEMENT
- PROJECT OBJECTIVES AND GOALS
- INVESTIGATOR BIOGRAPHIES
- RESEARCH PLAN SUMMARY
- PROJECT DELIVERABLES
- TIME LINE
- CONCLUSIONS TO DATE
- ISSUES/CHALLENGES
- BIBLIOGRAPHY
- ACRONYMS

PROBLEM STATEMENT

This project assesses the effectiveness of automated and manual vulnerability assessment methodologies in cloud and web-based applications. By comparing traditional tools with AI-driven solutions, the project aims to evaluate the accuracy, efficiency, and potential risks introduced by AI. Case studies and vulnerability detection tests will identify the strengths and limitations of each approach. The research will also explore how AI can be securely integrated into the assessment process, proposing improvements to enhance security in modern software environments.

PROJECT OBJECTIVES AND GOALS

Evaluate Automated Vulnerability Analysis Tools:

- Assess the limitations of automated tools like OWASP ZAP, Burp Suite, and Semgrep in identifying vulnerabilities in cloud and web-based applications.
- Perform detailed comparisons between automated and manual assessment methods, highlighting areas where each excels or falls short.

Analyze AI Integration in Vulnerability Assessments:

- Investigate how artificial intelligence can be leveraged to enhance vulnerability detection compared to traditional Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools.
- Evaluate AI's ability to address common vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Remote Code Execution (RCE).

Develop a Hybrid Approach to Security Testing:

- Propose new, hybrid methodologies combining AI-powered tools with traditional SAST/DAST techniques to create a more efficient and accurate vulnerability assessment process.
- Ensure these hybrid methodologies are securely integrated across the entire Software Development Life Cycle (SDLC).

Examine AI's Impact on Software Supply Chain Security:

- Analyze how vulnerability assessments, particularly AI-driven solutions, affect the security of software supply chains and their integration into modern cloud environments.

Measure Success with Defined Metrics:

- Establish evaluation criteria such as detection accuracy, false positive/negative rates, and adaptability to new threats.
- Use statistical methods and visual tools (e.g., bar charts, graphs) to validate findings and refine the proposed approaches based on performance data.

This project will provide a comprehensive analysis of automated, manual, and AI-driven vulnerability assessment techniques, proposing innovative solutions to improve security testing.

INVESTIGATOR BIOGRAPHIES

Nishanth Badim: Nishanth is leading the evaluation of automated vulnerability analysis tools. His expertise lies in performance benchmarking of automated solutions, particularly in real-world applications. He is responsible for coordinating the comparison between traditional Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools and newer AI-driven solutions. His in-depth knowledge of both traditional and automated methods ensures a balanced evaluation, identifying where AI can complement existing tools.

Nishanth brings expertise in cybersecurity and information technology, focusing on vulnerability analysis and secure software development. His work includes conducting manual vulnerability assessments and code reviews, aiming to balance automation with manual methods in security testing.

Harika Gattu: Harika focuses on researching the integration of Artificial Intelligence (AI) in vulnerability assessments. Her work centers on analyzing AI's potential to improve vulnerability detection and proposing new AI-driven approaches. Harika is also responsible for ensuring that AI implementations in the project follow security best practices throughout the Software Development Life Cycle (SDLC), mitigating risks introduced by AI while enhancing efficiency.

Harika specializes in network security and cloud infrastructure, bringing expertise in penetration testing and risk assessment. She focuses on the challenges of securing cloud and web-based applications and evaluating software supply chain security.

Divya-Bala-Sri Gogineni: Divya conducts case studies on real-world cloud and web-based applications to test and validate the chosen methodologies. She is responsible for documenting the findings from both manual and automated assessments, ensuring that the research covers practical application scenarios. Divya also assists in validating AI integration into the SDLC by identifying potential risks associated with AI-driven assessments.

Divya has a background in software engineering, focusing on secure coding practices and security testing. She is particularly interested in the combination of manual and automated testing methods for vulnerability assessments.

Bhavana-Bramaram Gurram: Bhavana oversees the comparison between manual and automated vulnerability assessments. She evaluates the limitations of manual methods and explores how AI can complement traditional approaches. In addition, Bhavana coordinates the writing and review of the final project report, ensuring that all findings and conclusions are clearly presented.

Bhavana is a software engineer with a keen interest in secure coding practices. She has experience in integrating AI into cybersecurity, focusing on AI-driven vulnerability assessments while addressing the risks involved in such integrations.

Savyasachi Kokkerala: Savyasachi leads the research on securely integrating AI into the SDLC. His focus is on proposing improved vulnerability assessment methodologies by combining AI and traditional techniques. Savyasachi ensures that all project findings align with industry best practices and security standards, contributing to the development of more robust and secure assessment strategies.

RESEARCH PLAN SUMMARY

Static Application Security Testing (SAST)

Static Application Security Testing (SAST) is a methodology that identifies security vulnerabilities by analyzing source code or compiled applications without executing them. SAST tools enable developers to detect potential weaknesses early in the Software Development Life Cycle (SDLC) by scanning source code, bytecode, or binaries for vulnerabilities like SQL Injection, Cross-Site Scripting (XSS), and buffer overflows.

When a vulnerability is discovered, SAST solutions generate detailed reports, offering code-level insights that allow developers to quickly pinpoint and resolve issues before the application is deployed to production. By continuously scanning throughout the development process, these tools help prevent security flaws from reaching live environments, reinforcing the overall security posture of the application.

SonarQube

SonarQube is a widely-used tool for continuous code quality analysis, designed to help development teams maintain clean and secure code. It automatically scans codebases to detect bugs, code smells, and security vulnerabilities across multiple programming languages.

Continuous Code Quality Analysis: SonarQube integrates seamlessly into CI/CD pipelines, ensuring that code quality is continuously monitored throughout the development process.

Bug Detection and Code Smell Identification: The tool identifies bugs and code smells early in the development cycle, allowing developers to improve maintainability and prevent future issues.

Vulnerability Detection: SonarQube helps developers spot potential security vulnerabilities, such as SQL Injection and XSS, before code is deployed, ensuring more secure applications.

Supports Multiple Languages: SonarQube provides support for a wide range of programming languages, making it versatile and applicable for various types of projects.

Integration with CI/CD Pipelines: By embedding SonarQube in CI/CD pipelines, teams can enforce code quality standards and ensure that each release is free of security risks and technical debt.

ShiftLeft

ShiftLeft is a security tool that offers real-time code analysis, designed to embed security checks directly into the development pipeline, ensuring early detection of vulnerabilities.

Real-Time Code Analysis: ShiftLeft analyzes code as it's written, providing immediate feedback on security issues, allowing developers to address them during coding.

Embedded Security Checks in Development Pipeline: By integrating security scans directly into the CI/CD pipeline, ShiftLeft ensures that vulnerabilities are caught early in the development process.

Early Vulnerability Detection: ShiftLeft prioritizes early identification of weaknesses, helping prevent vulnerabilities from advancing further into the development lifecycle.

Developer-Centric Security Insights: With detailed and actionable security insights tailored for developers, ShiftLeft fosters a security-first mindset, making it easier for teams to fix issues.

Continuous Feedback for Code Fixes: ShiftLeft offers continuous feedback loops, ensuring that developers receive timely suggestions for code improvements, making security a continuous process rather than an afterthought.

CodeQL

CodeQL is a powerful semantic code analysis tool designed to identify security vulnerabilities and logic flaws within codebases by running customizable security queries. It helps developers detect common vulnerabilities and ensure that code remains secure throughout the development process.

Semantic Code Analysis: CodeQL performs deep semantic analysis of code, understanding the logic and flow to detect vulnerabilities and flaws that might be missed by traditional static analysis tools.

Customizable Security Queries: With CodeQL, users can write and customize their own queries to identify specific security issues relevant to their codebase, providing flexibility in security analysis.

Detection of Logic Flaws and Vulnerabilities: CodeQL excels in finding complex logic flaws and vulnerabilities, such as SQL Injection and Cross-Site Scripting (XSS), which can be exploited by attackers.

Integration with GitHub for Continuous Scanning: CodeQL integrates directly with GitHub, enabling continuous security scanning as part of the development workflow, ensuring vulnerabilities are caught early and often.

Identifies Common Vulnerabilities: CodeQL includes built-in queries for detecting well-known vulnerabilities, like SQL Injection and XSS, making it an essential tool for maintaining secure applications.

GitGuardian Shield

GitGuardian Shield is a specialized security tool that focuses on detecting secrets and sensitive information, such as API keys and passwords, in code repositories. It prevents hardcoded secrets from being committed to source control and helps maintain secure DevOps practices.

Secrets Detection (API Keys, Passwords, etc.): GitGuardian Shield scans code for hardcoded secrets, like API keys and passwords, to prevent them from being accidentally exposed in repositories.

Real-Time Commit Scanning: The tool scans each commit in real-time to catch secrets before they are pushed to public or private repositories, ensuring sensitive data never leaves the developer's environment.

Prevents Hardcoded Secrets in Code Repositories: GitGuardian Shield ensures that hardcoded secrets are flagged immediately, preventing them from being committed and potentially exposed in production environments.

Alerts for Sensitive Information Leaks: The tool provides alerts when sensitive information is detected, enabling developers to act quickly and mitigate risks before they escalate.

Git Integration for DevOps Security: GitGuardian integrates seamlessly with Git and DevOps pipelines, embedding security directly into the version control system to enforce safe coding practices.

Testing Environment: Source Code

The testing platform for static security assessments leverages tools such as CodeQL, SonarQube, GitGuardian Shield, and ShiftLeft to scan and analyze source code for vulnerabilities before it's compiled or executed. These tools are designed to detect flaws early in the development lifecycle, providing an opportunity to address security gaps before the code reaches production. This proactive approach reinforces the overall security of the software by catching issues at the earliest stages.

CodeQL allows testers to create customized security queries, targeting vulnerabilities unique to the application's logic. It excels at identifying logic flaws, injection vulnerabilities, and other critical security issues. With GitHub integration, CodeQL performs continuous scanning of pull requests, ensuring vulnerabilities are detected and resolved as soon as they are introduced into the codebase.

SonarQube offers continuous code quality and security analysis by comparing source code against well-established security standards, such as the OWASP Top 10. It provides valuable insights into technical debt and security hotspots, ensuring that developers can maintain high-quality code while addressing potential security risks. SonarQube also enables long-term monitoring of the security posture across multiple projects.

GitGuardian Shield focuses on real-time detection of sensitive data within the codebase, such as hardcoded API keys, credentials, or tokens. Integrated directly into the Git workflow, it provides real-time alerts, helping developers prevent accidental exposure of sensitive information during the development process.

ShiftLeft integrates seamlessly into CI/CD pipelines, offering instant feedback as developers write code. Its "security as code" methodology allows developers to address vulnerabilities right at the source, as the code is being written. ShiftLeft also tracks vulnerabilities across releases, ensuring continuous security monitoring and remediation throughout the software development lifecycle.

By using this combination of SAST tools, development teams are equipped to proactively secure their codebases, catching vulnerabilities in real-time and safeguarding against potential security threats before the application reaches production.

Manual Testing Limitations

While manual review of source code offers deep insights into the logic and flow of an application, it comes with limitations:

- **Time-Intensive:** Manually inspecting source code for vulnerabilities can be incredibly time-consuming, especially for large codebases. Every line of code must be reviewed, which can slow down the overall development cycle.
- **Requires Expert Knowledge:** Manual SAST requires deep expertise in programming languages and security best practices. The effectiveness of manual reviews is highly dependent on the knowledge and experience of the tester, making it challenging to scale for larger teams or organizations.

Automation Limitations

While SAST tools automate code analysis, they also have their own challenges:

- **Limited to Static Code:** SAST tools analyze code at rest and are not capable of detecting vulnerabilities that may arise during runtime. This makes it difficult to detect issues like logic flaws that may only surface in specific scenarios when the application is running.
- **False Positives:** Like DAST tools, SAST tools can generate false positives, flagging issues that may not be critical or relevant to the actual security of the application. Sorting through these false positives can require significant time and effort.

AI Integrations in Static Security Testing

AI-powered solutions are becoming increasingly relevant in the realm of SAST, as they help to mitigate some of the limitations of manual and traditional automated tools:

- **Intelligent Vulnerability Detection:** AI-driven SAST tools can learn from large datasets of known vulnerabilities and continuously adapt to new threats. By recognizing patterns in code, AI tools can predict and identify novel security flaws that may not be part of predefined vulnerability databases.
- **Automated Security Queries:** AI integrations allow tools like CodeQL to automatically generate security queries based on common patterns seen across large codebases. This enhances the tool's ability to detect complex logic flaws that traditional automated tools may miss.

Dynamic Application Security Testing (DAST)

Dynamic Application Security Testing (DAST) is a security testing procedure designed to assess and analyze running applications by simulating penetration tests. Its primary goal is to uncover potential security vulnerabilities that could be exploited by malicious actors. Unlike static testing methods, which analyze code in a non-executing state, DAST tools interact with the application in real-time, assessing its behavior and responses during operation.

DAST tools are particularly effective in continuously monitoring web applications that are already in production, actively scanning for vulnerabilities that may arise during normal operation. They focus on identifying weaknesses, such as injection points or misconfigurations, that attackers could leverage to gain unauthorized access. By detecting these flaws, DAST solutions help organizations stay ahead of potential threats by highlighting how an attacker could remotely breach the system.

When a vulnerability is identified, DAST tools automatically generate alerts, notifying the appropriate teams so that the issue can be prioritized and addressed promptly. This real-time feedback loop allows for faster remediation and ensures that security teams can take swift action to mitigate potential risks, enhancing the overall security posture of the application.

OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is a popular open-source tool used for penetration testing of web applications. It is designed to identify and exploit potential security vulnerabilities by mimicking the actions of an attacker. ZAP offers a wide range of features that make it a comprehensive solution for dynamic testing, including:

Penetration Testing: ZAP simulates real-world attacks to expose potential weaknesses in a web application, helping security teams understand how their systems might be breached.

Application Spidering: This feature allows ZAP to systematically crawl the web application, mapping out its structure and identifying areas that may be vulnerable to attack.

Active and Passive Scanning: ZAP conducts both active and passive scans of web applications. Active scanning involves sending various payloads to the application to

detect vulnerabilities, while passive scanning observes the application's traffic without altering it, identifying issues like security misconfigurations or information leakage.

Fuzzing Techniques: ZAP can automatically inject various inputs into application fields to identify weaknesses like buffer overflows, SQL injections, or cross-site scripting (XSS). This helps uncover vulnerabilities that arise from improper input handling.

Session Hijacking Detection: ZAP is equipped to detect session hijacking vulnerabilities, where attackers might intercept or impersonate an active user's session, potentially gaining unauthorized access to sensitive data.

Nikto

Nikto is another open-source tool, focused on web vulnerability scanning, and is designed to detect known vulnerabilities in web servers and applications. It offers several features tailored for this purpose:

Web Vulnerability Scanning: Nikto performs comprehensive scans on web servers to uncover security issues such as outdated software, misconfigurations, or exposed directories.

Over 6,700 Known Vulnerability Checks: Nikto has a vast database of known vulnerabilities, allowing it to scan for over 6,700 potential weaknesses, providing security teams with detailed reports on their systems' security posture.

Outdated Software Detection: Nikto helps identify web servers running outdated or unsupported software versions, which could present significant security risks.

Customizable Scanning Options: The tool provides various customization options, allowing users to configure their scans based on specific security requirements or focus areas.

Compatible with Major Web Servers: Nikto is compatible with most major web servers, making it a versatile solution for organizations running a wide range of technologies. This flexibility ensures broad coverage and enhances its effectiveness in detecting security issues across different environments.

Testing Environment: OWASP Juice Shop

OWASP Juice Shop serves as the testing platform for conducting dynamic security assessments using ZAP and Nikto. Juice Shop is an intentionally vulnerable web

application designed for security training and testing purposes. It simulates a real-world e-commerce website, complete with common vulnerabilities found in many web applications, providing an ideal environment for penetration testing and vulnerability scanning.

By leveraging OWASP Juice Shop, testers can use **ZAP** for active and passive scanning, penetration testing, and fuzzing techniques, while simulating potential attacks such as session hijacking. This allows for a comprehensive exploration of vulnerabilities in a controlled setting.

Additionally, **Nikto** can be employed to perform extensive web vulnerability scanning on Juice Shop, checking for outdated software and performing over 6,700 known vulnerability checks, ensuring that the platform is effectively scrutinized for potential weaknesses.

The combination of Juice Shop, ZAP, and Nikto provides a robust testing environment, allowing for the identification of critical security flaws in real-time and demonstrating the practical use of DAST in real-world scenarios.

Manual Testing Limitations

While manual testing offers in-depth and targeted insights into an application's security, it presents several limitations:

- **Time-Consuming:** Conducting thorough manual penetration testing can be a lengthy process. Testers must manually inspect each part of the application, which requires a significant time investment, especially for large or complex systems.
- **High Expertise Required:** Manual security testing demands specialized knowledge and expertise in identifying and exploiting vulnerabilities. Testers must have a deep understanding of various attack vectors and the latest security trends, which can limit the scalability of the process if expert resources are scarce.

Automation Limitations

Automation tools like ZAP and Nikto help streamline the security testing process, but they also have inherent limitations:

- **Predefined Vulnerability Databases:** Automated tools rely heavily on predefined databases of known vulnerabilities. While they are effective at detecting common

security issues, they might miss novel or complex vulnerabilities not included in their database.

- **High False Positives:** Automated tools often produce false positives, flagging benign issues as security vulnerabilities. This can lead to unnecessary remediation efforts, wasting time and resources while diluting focus from genuine risks.

AI Integrations in Security Testing

To address the limitations of both manual and automated testing, AI integrations are becoming more common in security testing solutions:

- **Vulnerability Pattern Recognition:** AI-powered tools can analyze vast amounts of data and detect patterns that indicate potential vulnerabilities. These tools can identify novel security flaws by learning from past vulnerabilities and adapting to new attack methods.
- **Automated Exploit Generation:** AI can enhance the automation of exploit generation by automatically creating and testing payloads to exploit discovered vulnerabilities. This reduces the reliance on human intervention and speeds up the testing process, making it more efficient and comprehensive.

AI-Powered Penetration Testing: Deep Exploit

In our exploration of AI-driven security tools, we identified **Deep Exploit**, an advanced, fully automated penetration testing tool that leverages **Deep Reinforcement Learning (DRL)** to enhance the effectiveness of security assessments.

Deep Exploit stands out by:

- **Automating Exploit Execution:** Deep Exploit autonomously identifies and exploits vulnerabilities by assessing the status of all open ports on the target server. Using pinpoint accuracy, it executes the appropriate exploit, minimizing manual intervention and reducing time spent on vulnerability discovery.
- **Efficient Execution:** The tool efficiently selects and applies the most suitable exploits, optimizing resource usage and increasing the overall speed of penetration testing processes.
- **Deep Penetration & Self-Learning:** Unlike traditional tools, Deep Exploit utilizes self-learning capabilities through DRL, allowing it to adapt and refine its attack strategies over time. The more it interacts with a system, the better it

becomes at identifying vulnerabilities and successfully executing deep penetration attacks.

- **Fast Learning Time:** One of the significant advantages of Deep Exploit is its rapid learning curve. It quickly processes information and refines its approach, making it highly efficient for environments requiring swift and comprehensive testing.

By integrating AI with traditional penetration testing methodologies, Deep Exploit represents a significant step forward in the automation of security assessments, offering a powerful solution for organizations seeking faster, more reliable, and adaptive vulnerability detection.

Project Deliverables

1. Final Report:

- **Research Methods:** Outline the methodologies used for evaluating SAST and DAST tools, including data collection and analysis techniques.
- **Key Findings:** Summarize the most significant insights from comparing the tools, focusing on performance, effectiveness, and areas for improvement.
- **Actionable Insights:** Provide recommendations on how to effectively implement the tools and suggestions for improving vulnerability assessments.

2.Presentation Materials: presentations for both academic and industry audiences, focusing on methodologies for the academic side and practical applications for industry professionals.

3.Tool Comparison Metrics: Develop criteria to evaluate the effectiveness of different SAST and DAST tools, focusing on performance, usability, and integration with AI-based enhancements.

4.Future Recommendations:Suggest ways to improve AI integration in vulnerability assessment tools and areas for further research and development.

5.Resources and Guidelines:Provide documentation and resources to help others validate and extend the project outcomes, including guidelines for replicating the tool assessments.

TIMELINE

Sep 15 – Project Planning: Defined the project scope, objectives, and deliverables. This phase involved outlining the research methodologies for evaluating SAST and DAST tools, setting goals for AI integration, and determining success criteria for the tool comparison metrics.

Sep 29 – Environment Setup: Completed the setup of the technical environment needed for the project, including installing the necessary software tools, setting up cloud environments or virtual machines, and ensuring that all dependencies for running SAST and DAST tools were in place. This phase also included configuring the development environment for conducting vulnerability assessments.

Oct 6 – Tool Configuration: Configured the selected SAST and DAST tools for assessment. This step involved integrating the tools into the testing environment, calibrating the settings to ensure optimal performance for testing different codebases, and running initial tests to verify tool functionality.

Oct 27 – Vulnerability Assessment: The project will move into the vulnerability assessment phase, where the configured SAST and DAST tools will be used to conduct detailed vulnerability scans. Data on detected vulnerabilities will be collected, and the tools' performance will be evaluated based on criteria like accuracy and speed.

Nov 10 – Data Analysis: During this phase, the collected data will be analyzed to compare the effectiveness of the SAST and DAST tools. The analysis will focus on identifying the strengths and weaknesses of each tool and evaluating the potential impact of AI integration on vulnerability assessment.

Nov 24 – Report Preparation: The key findings from the assessments and analysis will be compiled into a detailed final report. This will include actionable insights, recommendations for tool improvements, and suggestions for future work on AI-enhanced vulnerability assessment tools.

Dec 6 – Presentation: The final presentation will be prepared and delivered to both academic and industry audiences. It will highlight the project's methodologies, key findings, and recommendations, with an emphasis on practical applications and potential research contributions.

CONCLUSIONS TO DATE

As part of the ongoing research, an in-depth analysis of various SAST and DAST tools has already revealed key insights into the strengths and weaknesses of both automated and manual approaches. Automated tools demonstrate advantages in scalability and speed, allowing for quick vulnerability detection across large codebases. However, manual approaches continue to provide a more nuanced, contextual understanding that is critical in identifying complex vulnerabilities.

Preliminary findings suggest a scarcity of open-source tools with robust AI integration. While AI shows significant potential for improving vulnerability detection and reducing false positives, further research is needed to explore these capabilities in depth.

Additionally, areas where AI could significantly enhance vulnerability assessment have been identified, though further investigation is required to validate these findings and explore opportunities for innovation. The research will continue to explore these gaps and evaluate the potential for future developments in AI-driven security tools.

ISSUES/CHALLENGES

Tool Selection and Compatibility: Selecting the right SAST and DAST tools that align with the project's scope and the technologies in use has proven challenging. Achieving comprehensive coverage is difficult, especially when attempting to balance the need for accurate detection with the capabilities of available tools.

Resource Demands for Manual Testing: Manual testing requires significant resources, including specialized expertise and time. This can create bottlenecks in the testing process, potentially delaying the overall project timeline as manual approaches are both labor-intensive and time-consuming.

Configuration Challenges: Automated tools must be configured precisely to deliver accurate results. Ensuring that these tools provide thorough security coverage across all system components involves a delicate balance of settings and parameters, which can be difficult to get right.

Expertise Dependency: The effectiveness of manual testing is highly dependent on the skill level of the testers. This reliance on expertise introduces variability, increasing the potential for human error and inconsistencies in the detection of vulnerabilities.

BIBLIOGRAPHY

- NIST, NIST Special Publication 800-53 Revision 5: Security and Privacy Controls for Information Systems and Organizations. National Institute of Standards and Technology, Sept. 2020. Available: <https://doi.org/10.6028/NIST.SP.800-53r5>
- OWASP. (2023, February). OWASP Top Ten 2023. OWASP Foundation. Available: <https://owasp.org/www-project-top-ten/>
- <https://owasp.org/site-documentation/>
- CISA. (2022, July 14). Mitigating Log4Shell and Other Log4j-Related Vulnerabilities. CISA. Available: <https://www.cisa.gov/uscert/ncas/alerts/aa22-014a>
- Alazab, M., Tang, M., & Choo, K.-K. R. (2019). Artificial Intelligence in Cybersecurity: A Bibliometric Analysis of the Current State of Research. IEEE Access, 7, 101759–101772. <https://doi.org/10.1109/ACCESS.2019.2929938>
- Veracode. (2022, November). The State of Software Security: Securing Software Supply Chains. Veracode. Available: <https://www.veracode.com/sites/default/files/pdf/resources/reports/stateof-software-security-2022.pdf>

ACRONYMS

AWS: Amazon Web Services – A comprehensive cloud platform providing a wide range of services like computing power, storage, databases, and machine learning, enabling scalable and cost-effective solutions for businesses.

DAST: Dynamic Application Security Testing – A method of testing the security of an application by examining it in its running state for vulnerabilities.

RCE: Remote Code Execution – A security vulnerability that allows an attacker to run arbitrary code on a remote machine.

SAST: Static Application Security Testing – A method of analyzing source code or compiled applications for vulnerabilities without executing the code.

SDLC: Software Development Life Cycle – A process for planning, developing, testing, and deploying software systems, typically involving stages like requirements, design, coding, testing, and maintenance.

XSS: Cross-Site Scripting – A type of security vulnerability where an attacker injects malicious scripts into web pages viewed by others.